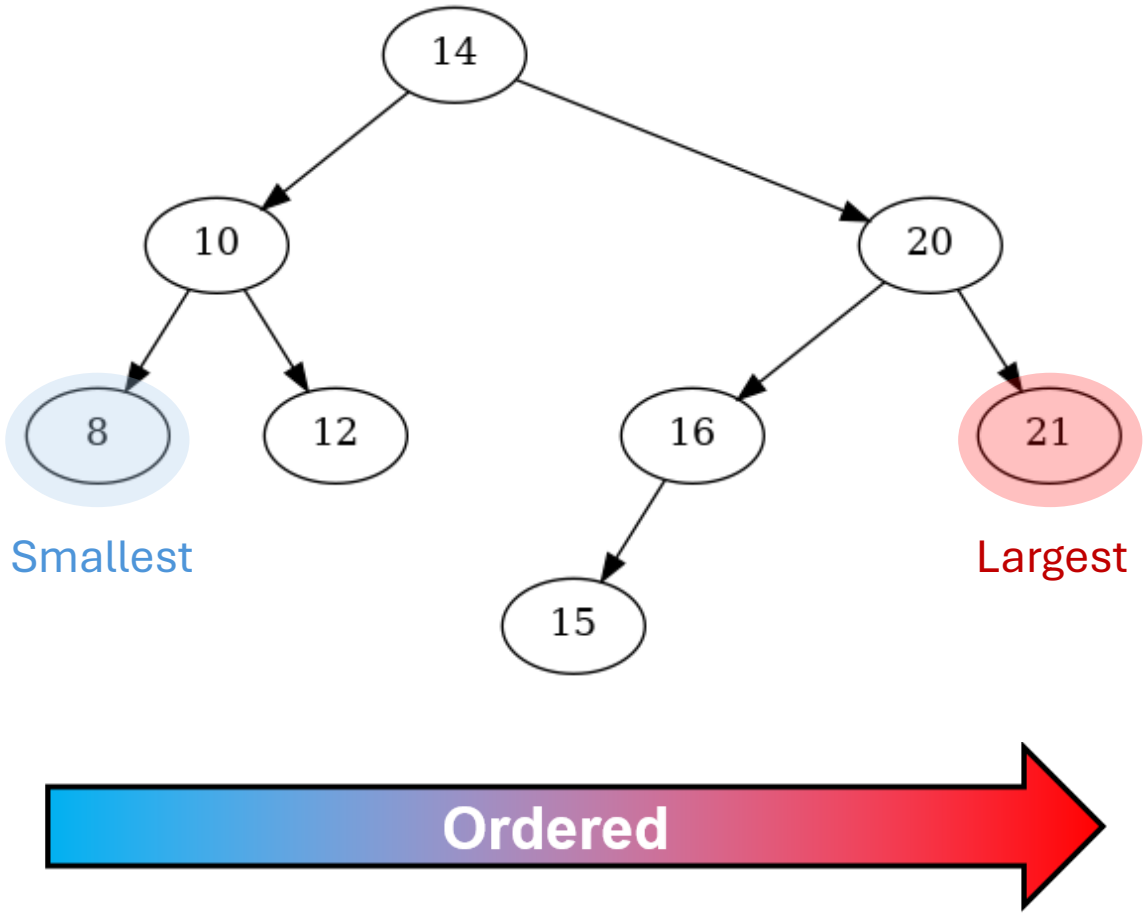
Binary Search Tree

Binary Search Tree



Binary:

- Every node can have at most 2 children: left and right.

Search:

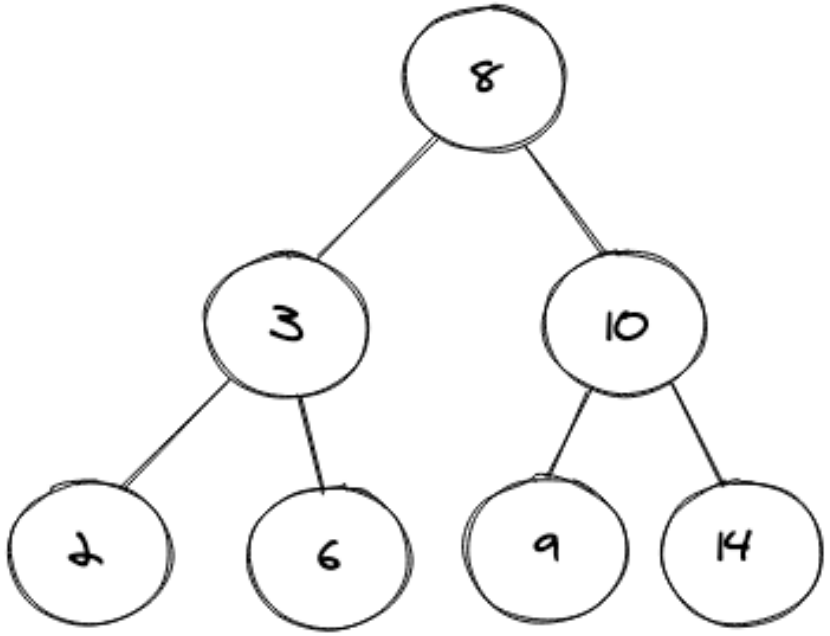
- For every node, the left subtree always contains values less than the node.
- For every node, the right subtree always contains values greater than the node.

Balanced BST

- The tree has to be balanced for this O(log n) complexity to apply

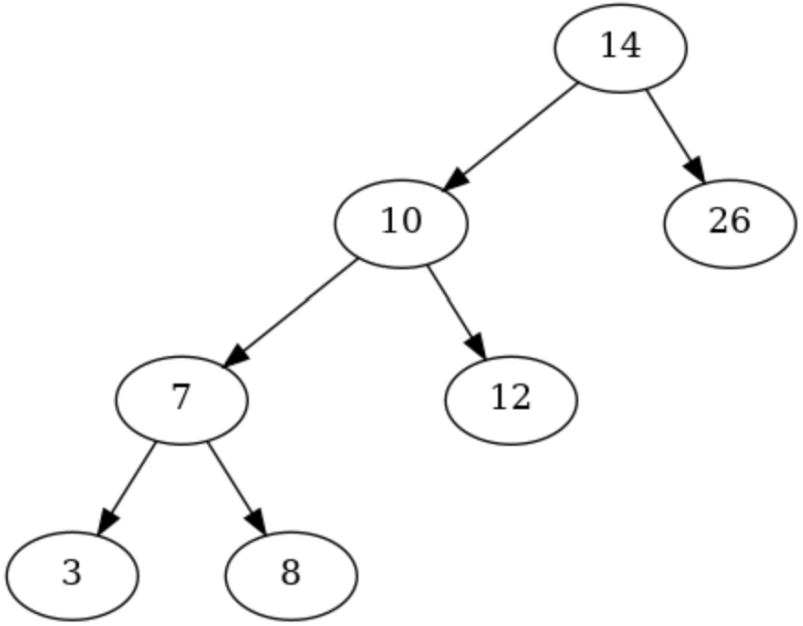
- A BST is **balanced** if the **height of the left and right subtrees** of every node differ by **at most** 1.

- This keeps the tree roughly "even" so it doesn’t become skewed.



More With Trees

Is This Tree Balanced?



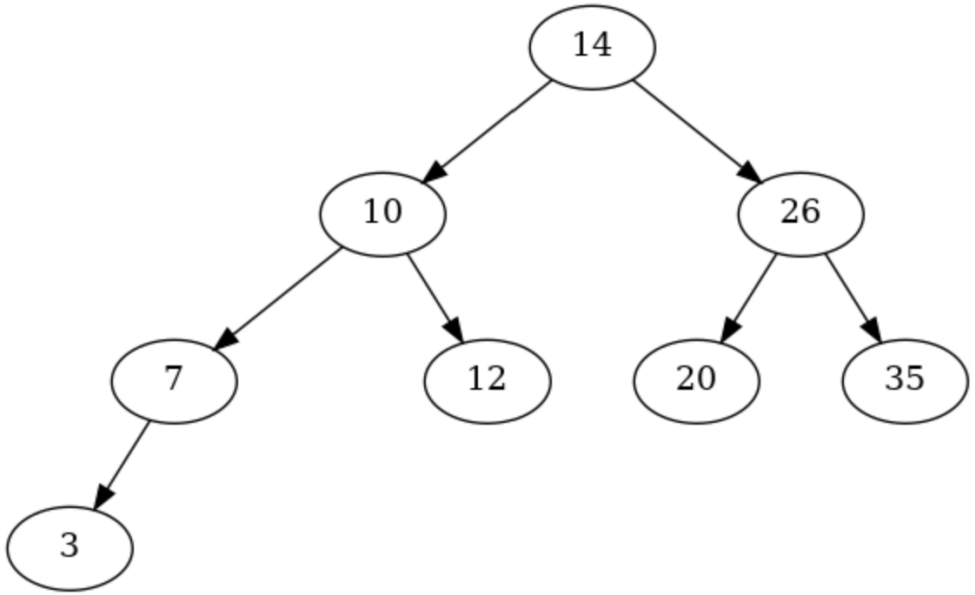
It's Called a **Full Tree**

Every node has
0 OR 2 children

More With Trees

Is This Tree Balanced?

Is This Tree Full?

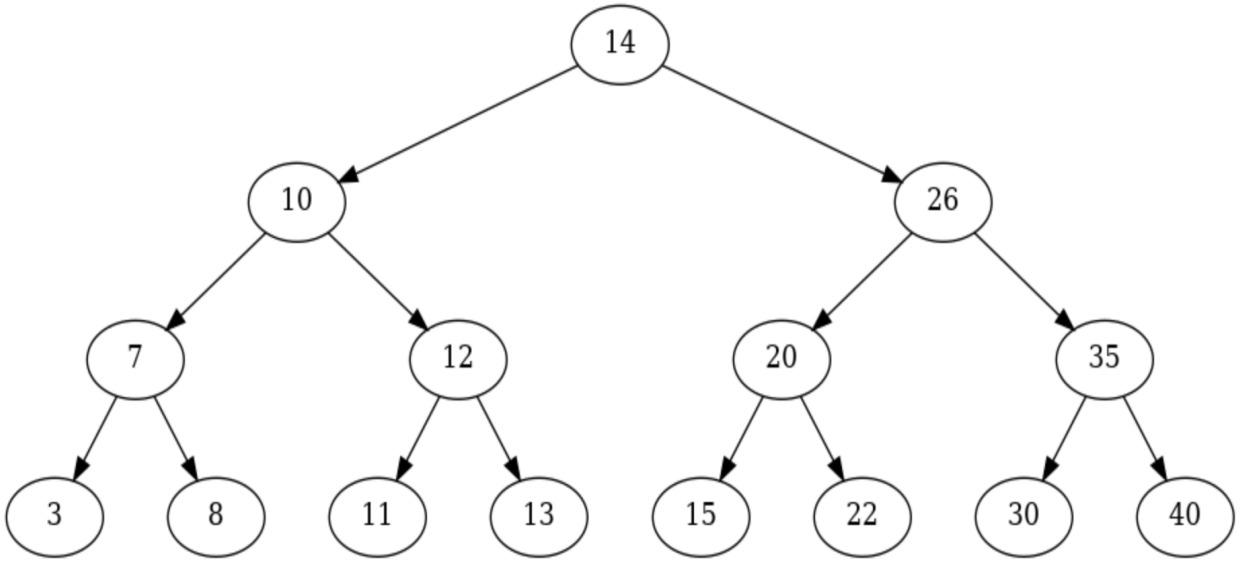


We call it a **Complete Tree**

1) **Every level** is **Full** except the last level
2) **Last level** has all **nodes** as **far left** as possible

More With Trees

What if we fill it all in ....

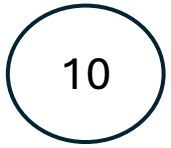


This is called **Perfect Tree**

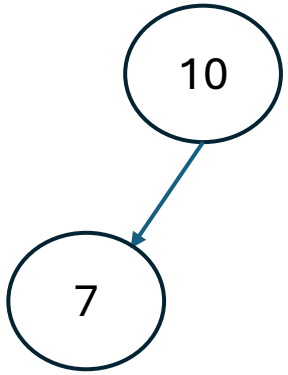
All levels are Full

More With Trees

What is the simplist Perfect Tree?



Is this full?



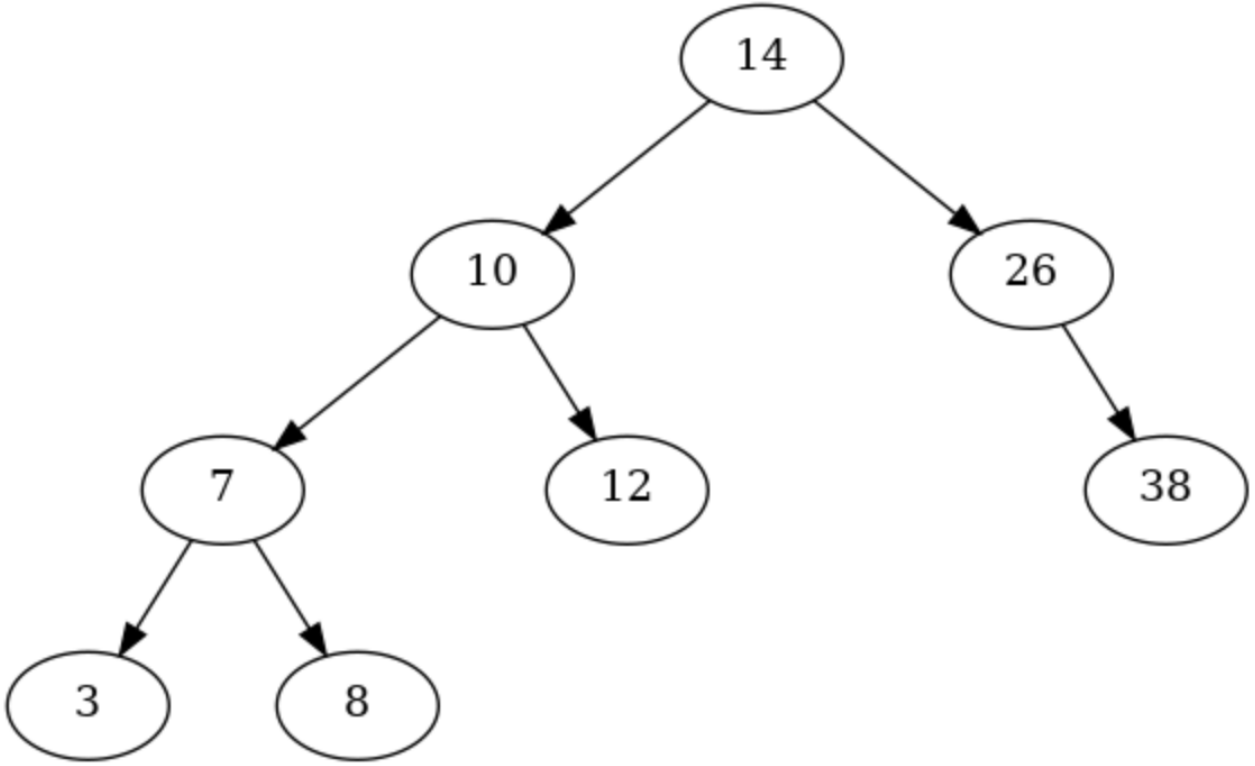
Are all Full Trees Perfect?

Are all perfect Trees Full?

Are all Complete Trees Perfect?

Are all Perfect Trees Complete?

More With Trees



Full?

Complete?

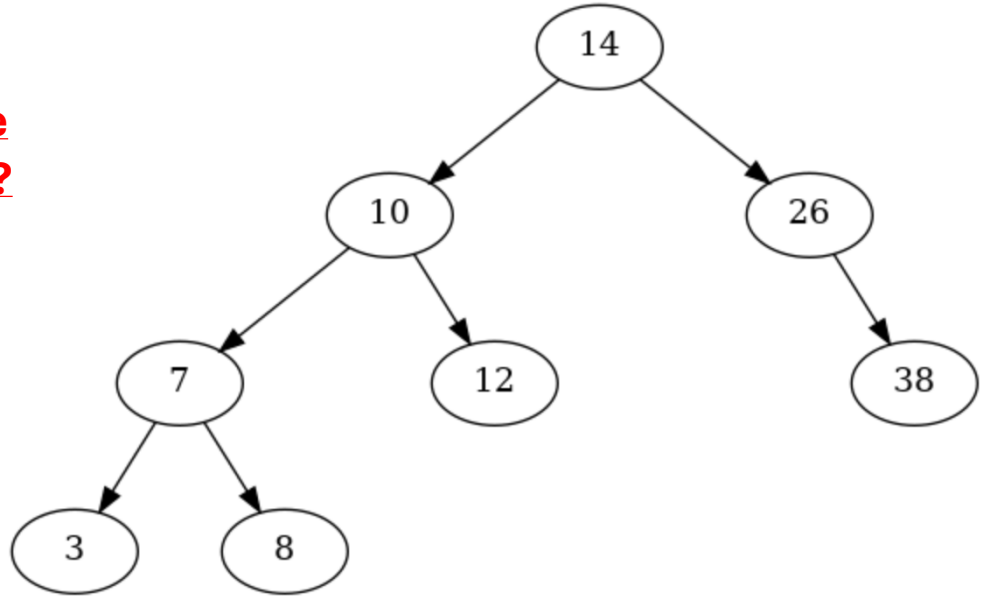
BST?

Representing Trees With Arrays

How many array slots do we need for a tree with heigh h?

A tree with heigh **h**, will has $2^h - 1$ nodes

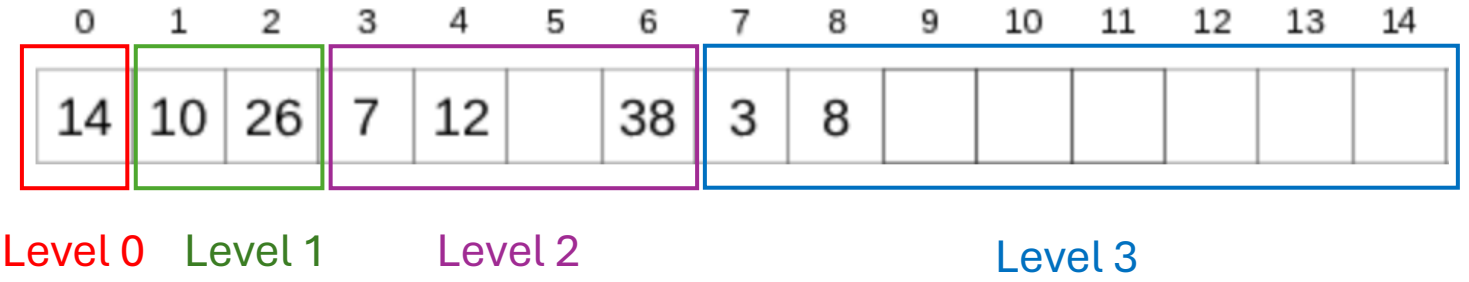
So **array length** will be $2^h - 1$



For a node at index (i),

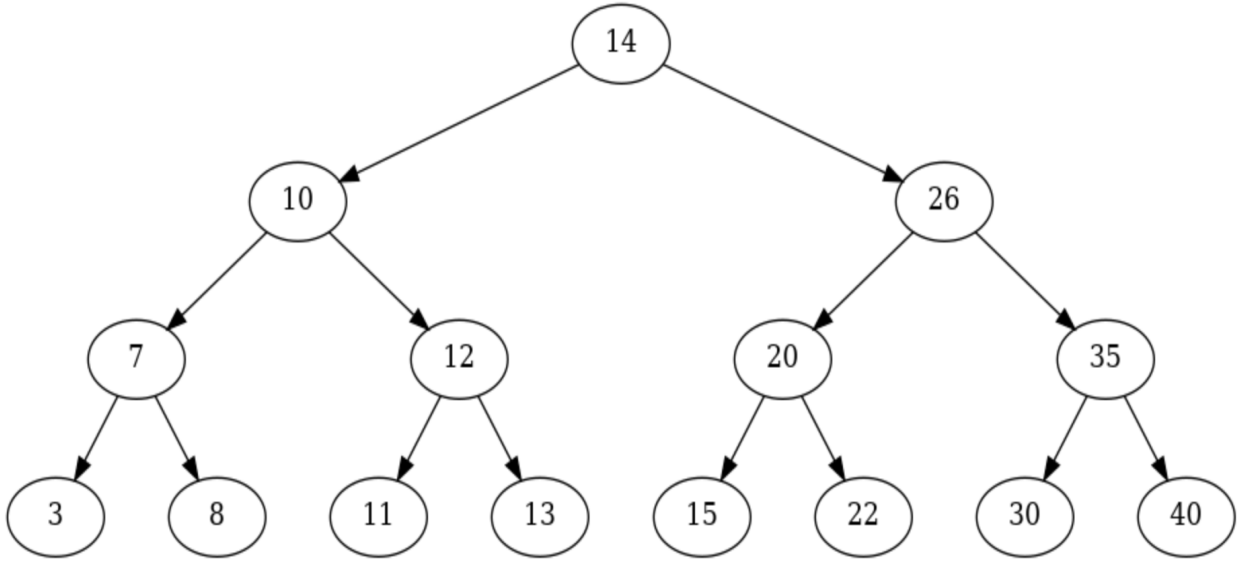
Where's its left child located?

Where's its right child located?



Representing Trees With Arrays

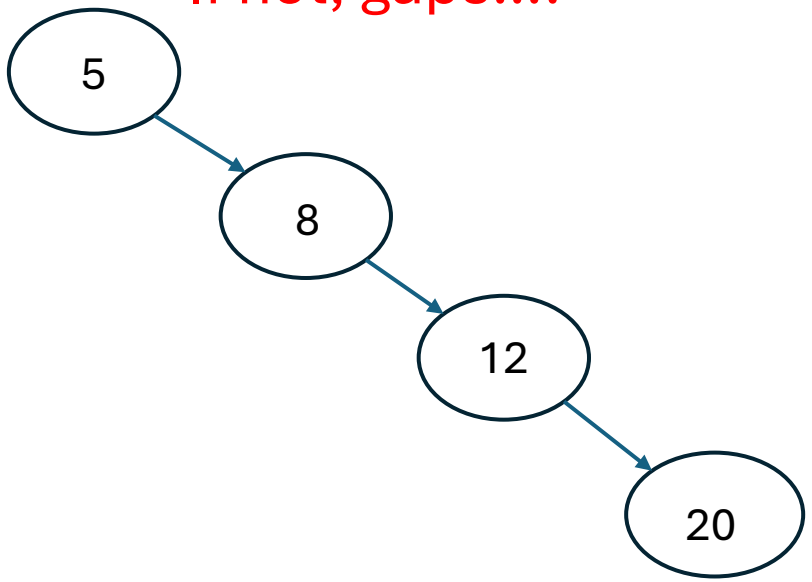
If the tree is perfect, Array is full



Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Value	14	10	26	7	12	20	35	3	8	11	13	15	22	30	40

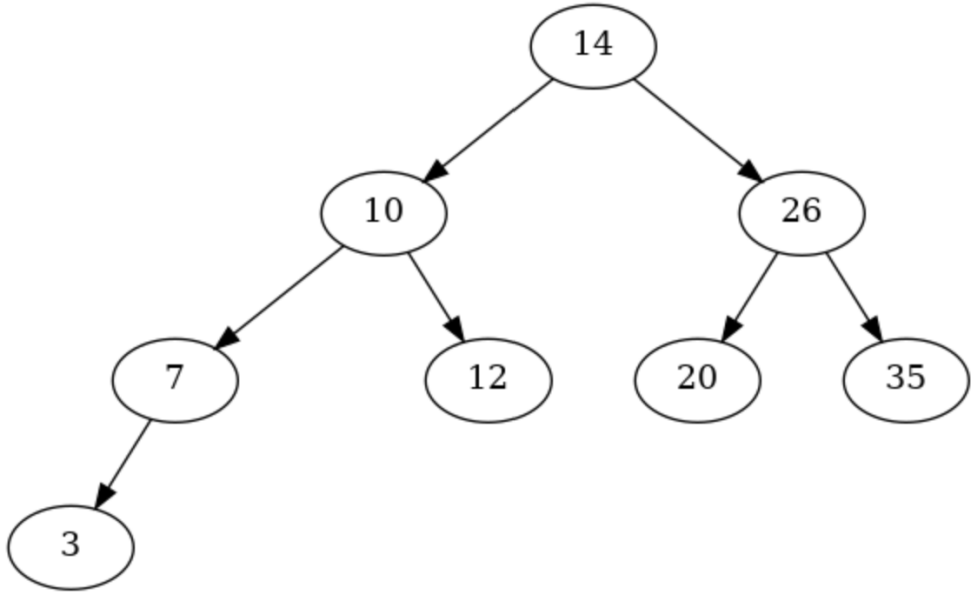
Representing Trees With Arrays

If not, gaps....



Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Value	5		8				12								20

Representing Trees With Arrays



Good for trees that are complete or nearly complete.

Access nodes through index calculations (no pointer tracking)
E.g. Min and Max items

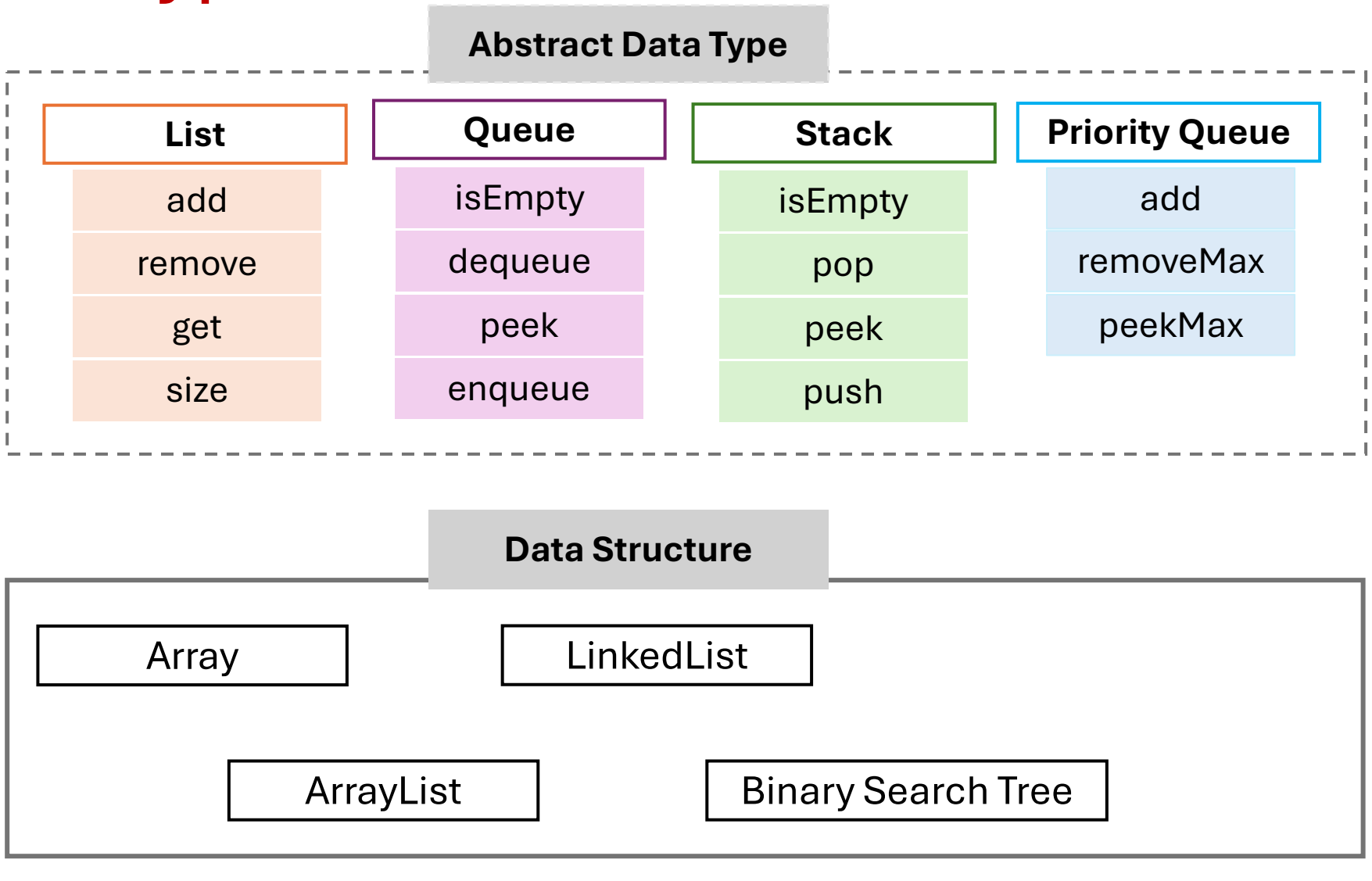
Priority Queue



Abstract Data Type & Data Structures

Contract: formal description of the behavior of a data type (*What Operations The Data Type Has?*)

Implementation: (*How Are These Operations Implemented?*)



Priority Queue – Example Use Cases

- **To-do list**
- **Operating-system process scheduling**
- **Network traffic routing**
- **Emergency Rooms**
- **Shortest Path Algorithms** (later in the course)

Priority Queue Implementations - ArrayList

- Unsorted
  - Add 3,7,1,5
  - RemoveMax
- Sorted
  - Add 3,7,1,5
  - RemoveMax
- Which One is Better to Use?

Priority Queue Implementations - LinkedList

- Unsorted
  - Add 3,7,1,5
  - RemoveMax
- Sorted
  - Add 3,7,1,5
  - RemoveMax
- Which One is Better to Use?

Priority Queue Implementations - BST

- Add 3,7,1,5
- RemoveMax

Priority Queue Implementations

Implementation	add	peekMax / removeMax	Why
Array, unordered	O(1)	O(N)	Append at the end, but you must scan everything to find the max
Array, sorted	O(N)	O(1)	Shift items to insert in order; the max sits at the end. A really good O(N): shifting in an array is very fast on modern hardware.
Linked list, unordered	O(1)	O(N)	Add at the head; scan to find the max
Linked list, ordered	O(N)	O(1)	Walk to the right spot to insert; the max is at the head
BST	O(log N)	O(log N)	Max is the rightmost node. Only if balanced, which ours doesn't guarantee